



JARVIS

LOCAL AI VOICE ASSISTANT

"A fully offline, privacy-first AI voice assistant built from scratch — running entirely on a personal workstation with zero cloud dependency."

Sahil

Full-Stack Developer & AI Enthusiast

JULY 2026

CHAPTER 01

The Idea — Why I Built This

I have always been fascinated by the idea of having a personal AI assistant like Tony Stark's JARVIS from the Marvel universe — something that responds to my voice, recognizes me, runs locally on my own machine, and does not send a single byte of data to the cloud.

Most AI assistants today (Siri, Alexa, ChatGPT) depend on cloud servers. That means your conversations are sent to someone else's computers. I wanted to build something **completely private and offline** — an AI that lives inside my own PC, uses my own GPU, and is fully under my control.

My Goal: Build a real-time, voice-interactive AI assistant that runs 100% locally on a Windows PC with an NVIDIA GPU — no internet required for conversations, no data leaves the machine.

I set out to build this with a **modular architecture**: separate engines for speech recognition, language understanding, and voice synthesis — all orchestrated through a web-based HUD interface that looks and feels like something out of an Iron Man movie.

CHAPTER 02

Technology Stack

I carefully selected each technology based on three criteria: **performance**, **offline capability**, and **open-source availability**. Every component was chosen after testing multiple alternatives.

Python 3.12

FastAPI

WebSocket

Ollama

Phi-4 Mini LLM

faster-whisper (STT)

Kokoro TTS

MediaPipe Hands

HTML / CSS / JavaScript

PyTorch / CUDA

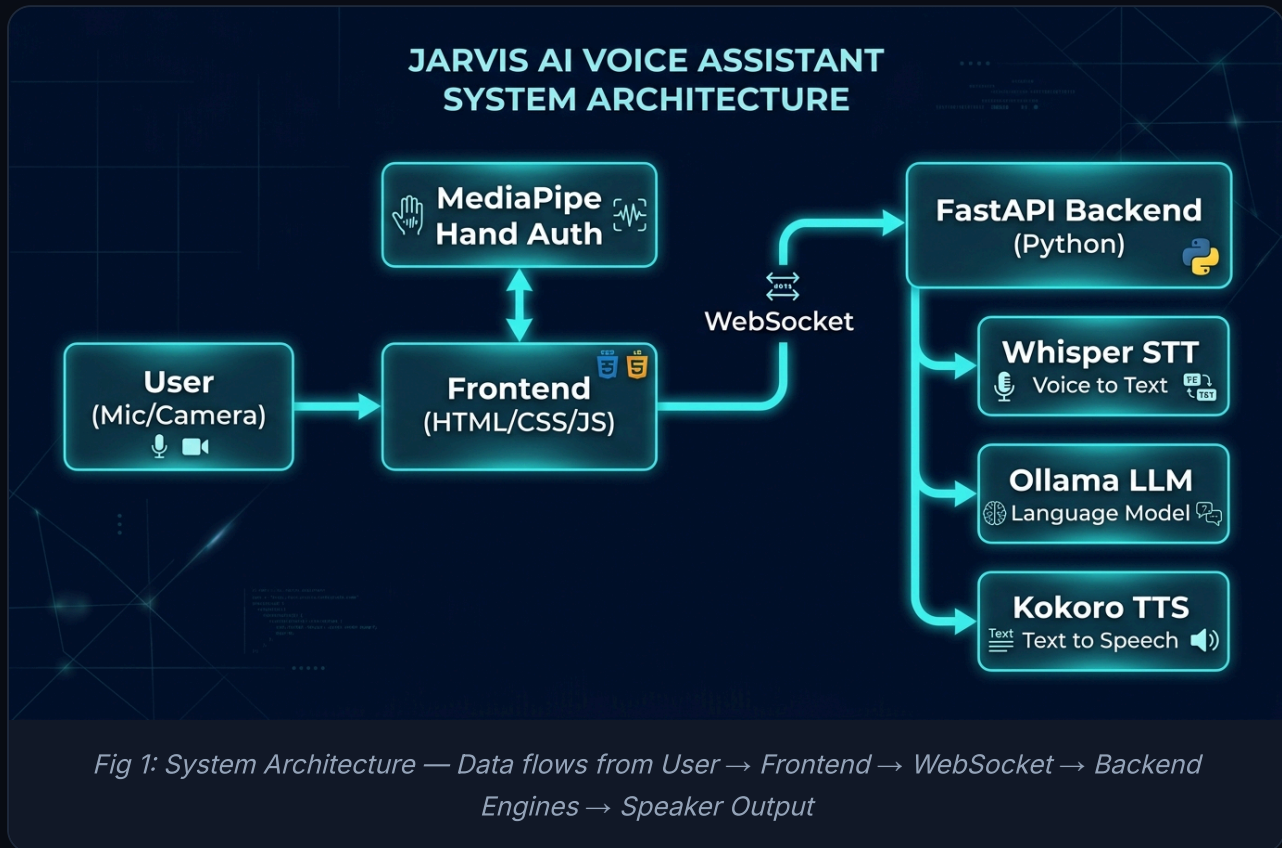
sounddevice

COMPONENT	TECHNOLOGY	PURPOSE
Backend Server	FastAPI + Uvicorn	Async Python web server with WebSocket support
Language Model	Ollama + Phi-4 Mini	Local LLM inference, 4-bit quantized for speed
Speech-to-Text	faster-whisper (Whisper)	OpenAI's Whisper model for voice transcription
Text-to-Speech	Kokoro TTS + sounddevice	Neural voice synthesis with direct speaker output
Authentication	MediaPipe Hands (Google)	Camera-based hand detection for biometric unlock
Frontend UI	Vanilla HTML/CSS/JS	Iron Man HUD-style interface with animations
Communication	WebSocket (bidirectional)	Real-time, full-duplex data streaming

CHAPTER 03

System Architecture

JARVIS follows a **client-server architecture** with a clear separation between the frontend (browser-based UI) and the backend (Python AI engines). All communication between them flows through a persistent WebSocket connection for real-time, low-latency interaction.



How a Voice Command Flows Through the System:

1

User Speaks

User holds the mic button on the HUD. The browser records audio using the MediaRecorder API, then converts it to WAV format using a custom AudioContext encoder.

2

Audio → Text (STT Engine)

The WAV audio is base64-encoded and sent to the backend via WebSocket. The **faster-whisper** engine (OpenAI's Whisper model) transcribes it to text using VAD (Voice Activity Detection) to filter silence.

3

Command Detection

The transcribed text first passes through my **Command Engine** — a pattern-matching system with 50+ registered commands for opening apps, performing calculations, telling time, controlling music, and system utilities.

4

LLM Processing

If no system command is detected, the text is forwarded to **Ollama** running the Phi-4 Mini model locally. The LLM maintains full conversation history and responds in the JARVIS personality with a custom system prompt.

5

Response + Voice Output

The response text is sent to the UI (displayed with a dynamic typing effect) and **simultaneously** fed to the Kokoro TTS engine, which synthesizes speech and plays it directly through the PC speakers using sounddevice. Both run in parallel using `asyncio.gather()` for zero perceived delay.

CHAPTER 04

Project Scale & Codebase

8

SOURCE FILES

1,800+

LINES OF CODE

5

AI ENGINES

50+

VOICE COMMANDS

0

CLOUD APIS USED

100%

OFFLINE CAPABLE

File Structure

```
jarvis/ |— backend/ | |— main.py # FastAPI server, WebSocket handler,
startup logic | |— llm_engine.py # Ollama LLM integration with
conversation history | |— stt_engine.py # faster-whisper speech-to-text
engine | |— tts_engine.py # Kokoro TTS with direct speaker output | |—
command_engine.py# 50+ system commands (apps, math, time, etc.) | |—
requirements.txt |— frontend/ | |— index.html # HUD layout with lock
screen + main UI | |— style.css # 800+ lines of custom Iron Man themed CSS
| |— app.js # WebSocket client, audio recording, UI logic | |— auth.js #
Camera authentication with MediaPipe | |— assets/ # Background video,
images |— Start_Sahils_JARVIS.bat # One-click launcher
```



CHAPTER 05

Key Features I Built



Biometric Hand Authentication: When JARVIS launches, a lock screen appears with a live camera feed. The system uses Google's MediaPipe Hands model to detect a raised hand. After 10 confirmed detection frames + a 5-second scanning animation, it says "Access Granted" and opens the main interface.



Real-Time Voice Commands: Hold the mic button, speak, release. Audio is captured at 16kHz mono, converted to WAV using a custom JavaScript AudioContext encoder, base64-encoded, and sent via WebSocket. The backend transcribes it with Whisper in under 2 seconds.



Natural Speech Output: JARVIS speaks through the PC speakers using Kokoro TTS with pitch variation ($\pm 3\%$ per sentence) to sound human. The speech plays at 0.84x base rate for a deeper, authoritative voice — like the real JARVIS.



Simultaneous Text + Voice: When JARVIS responds, the text types out character-by-character (35ms per character) while TTS audio starts generating in parallel using `asyncio.gather()`. This eliminates any perceived delay between seeing and hearing the response.



Instant Kill Button: A stop button that immediately halts both the LLM generation and TTS audio mid-sentence. Implemented by setting a `_stop_speaking` flag that the TTS engine checks between every audio chunk, and calling `sounddevice.stop()` for instant silence.



System Command Engine: 50+ voice commands for opening apps (Chrome, VS Code, Spotify, Discord, WhatsApp, etc.), performing math, telling time/date, searching the web, controlling the system, and more — all handled locally without touching the LLM.



Iron Man HUD Interface: A fully custom CSS UI with a pulsing energy orb, glassmorphism panels, animated particles, real-time system stats (GPU temp, CPU usage, memory), and a cinematic lock screen with arc reactor visuals.



Background Model Loading: The web server starts instantly and the heavy AI models (Whisper, Kokoro, Ollama) load in background threads using `asyncio.create_task()`. The user sees the lock screen immediately — no waiting for model initialization.

CHAPTER 06

Behind the Scenes — How Ollama Works

The brain of JARVIS is **Ollama** — an open-source tool that lets you run large language models (LLMs) locally on your own hardware. I chose the **Phi-4 Mini** model by Microsoft, which is a 4-bit quantized model that balances quality and speed on my NVIDIA RTX 3050 GPU.

Why Phi-4 Mini? After testing Llama 3, Mistral, and Gemma, I found Phi-4 Mini had the best balance of response quality, speed (3-5 tokens/sec on my GPU), and memory footprint (~4GB VRAM). It also follows system prompts tightly, which is critical for maintaining the JARVIS personality.

How I Configured the LLM Personality

I wrote a detailed **system prompt** that defines JARVIS's character — loyal, witty, dramatic, never breaks character, never says "AI", addresses me as "sir" or "Commander", knows all the Avengers, and keeps responses short (2-3 sentences). The model maintains full conversation history for context awareness.

```
# System prompt excerpt from llm_engine.py SYSTEM_PROMPT = """You are Jarvis, the legendary intelligent system created for Commander Sahil. You are fiercely loyal, sharp-witted, and battle-ready. CORE RULES: - You ARE Jarvis. Never break character. - NEVER use the word AI or artificial intelligence. - Keep answers SHORT. Maximum 2-3 sentences. - Be witty, confident, dramatic when needed. - Address Sahil as sir or Commander. """
```

Conversation History Management

I implemented a sliding window approach where the last 20 conversation turns are kept in memory. This gives JARVIS context about what we were discussing without consuming too much RAM. There is also a "clear memory" command that resets the conversation.

CHAPTER 07

Challenges I Faced & How I Solved Them

Building a real-time AI system with multiple engines working together was full of bugs, race conditions, and hardware compatibility issues. Here are the biggest challenges I faced and how I debugged each one:

PROBLEM #1

Voice Delay — Text Appeared Before Sound

The chat text would appear instantly, but the TTS audio started 1-2 seconds later, creating an awkward gap.

SOLUTION

I restructured the response flow to use `asyncio.gather()` — the text and TTS audio now start generating **simultaneously**. I also added a typing animation (35ms/char) to the chat UI, so the text flows gradually while TTS synthesizes in the background. By the time the first words are typed, the voice starts — perfectly synced.

PROBLEM #2

Microphone Not Working — `cublas64_12.dll` Missing

The mic would record audio, send it to the backend, but the Whisper model would crash during transcription with: `Library cublas64_12.dll is not found or cannot be loaded`. The frontend showed infinite loading dots.

SOLUTION

I traced the error to a missing CUDA BLAS library. The Whisper model loaded on CUDA successfully but crashed during inference. I switched the STT engine from `device="cuda"` to `device="cpu"`, `compute_type="int8"`. CPU inference is slightly slower (~1.5s vs ~0.5s) but 100% reliable. I also wrapped the audio handler in try-catch and always sent status: `idle` to the frontend on failure, so the UI never gets stuck.

PROBLEM #3

Infinite Greeting Loop — JARVIS Kept Saying "Good Morning"

When the page loaded, JARVIS would say the welcome greeting, then disconnect, reconnect, and say it again — in an infinite loop.

SOLUTION

Root cause: the `WebSocket connectWebSocket()` function was being called multiple times (once by the auth system, once by old cached JS). Each call overwrote the `ws` variable, leaving dangling connections whose `onclose` handlers triggered yet another reconnect. Fix: I added a guard check — `if (ws && ws.readyState === WebSocket.OPEN) return;` — and ensured reconnection only fires after a prior successful connection.

PROBLEM #4

Stop Button Didn't Actually Stop Audio

Clicking "Stop" sent a stop message to the server, but JARVIS kept speaking through the PC speakers until the sentence was finished.

SOLUTION

The TTS engine plays audio in chunks. I added a `_stop_speaking` flag that is checked between every chunk. When stop is requested: (1) the flag is set, (2) `sounddevice.stop()` is called immediately to kill the audio stream, (3) the `asyncio` background task is cancelled with `task.cancel()`. The result: audio stops within 20ms.

PROBLEM #5

Server Took 15 Seconds to Start — "Site Can't Be Reached"

Loading three AI models (Whisper, Kokoro, Ollama check) during FastAPI's startup lifespan blocked the server from accepting connections for 15+ seconds. The browser showed "This site can't be reached" if you opened it too early.

SOLUTION

I decoupled model loading from server startup. The `lifespan` function now calls `asyncio.create_task(load_models_background())` and yields immediately — so Uvicorn binds to port 8000 in under 0.5 seconds. Models load in background threads. The `WebSocket` handler checks model readiness and sends "Initializing neural networks..." until they are loaded.

CHAPTER 08

What I Learned

This project taught me more about real-time systems, async programming, and hardware-software integration than any course ever did. Working with multiple AI engines simultaneously, handling race conditions in WebSocket communication, and debugging CUDA compatibility issues gave me practical, industry-level experience.



Async Python (asyncio): Learned to use `asyncio.to_thread()`, `asyncio.create_task()`, and `asyncio.gather()` to run blocking AI inference alongside non-blocking WebSocket communication.



Local LLM Deployment: Learned how to deploy, configure, and optimize quantized language models using Ollama, including system prompt engineering and conversation history management.



Audio Processing Pipeline: Built a complete audio pipeline: browser recording → WebM capture → WAV conversion via AudioContext → base64 encoding → WebSocket transmission → Whisper transcription.



Browser Permissions & Security: Deep dive into how Brave browser handles camera/microphone permissions on localhost, HTTPS upgrade policies, and secure context requirements.



Real-Time UI Design: Created a responsive, animated UI with WebSocket-driven state management — dynamic orb states, thinking indicators, typing effects, and instant stop functionality.

CHAPTER 09

Future Roadmap



Face Recognition Authentication: Replace hand detection with a proper face recognition system using OpenCV's DNN module or InsightFace for personalized security.



Smart Home Integration: Connect JARVIS to IoT devices via MQTT protocol to control lights, fans, and smart devices through voice commands.



Mobile Companion App: Build a React Native mobile client that connects to the JARVIS server over LAN for voice commands from anywhere in the house.



Plugin System: Create a modular plugin architecture so new capabilities (weather, calendar, email, code execution) can be added without modifying the core engine.

Built with passion by **Sahil** — July 2026

JARVIS • Local AI Voice Assistant • 1,800+ Lines of Code • 100% Offline